

Prise en main du flow numérique ASIC



Romain DUCHADEAU (IP2I / CNRS) ¹

Dernière modification:

08/09/2026

1. r.duchadeau@ip2i.in2p3.fr

Table des matières

1. Introduction	4
1.1. Objectif	5
1.2. Arborescence du dossier du tutoriel	6
1.3. Approche pédagogique : méthode en plusieurs niveaux d'abstraction	6
1.4. Informations pratiques	7
2. Etape 1 : Simulation RTL - outil Xcelium	8
2.1. Entrées et sorties	8
2.2. Niveau 1 : Commande Xcelium minimale	8
2.2.1. Définition du contrat logique	9
2.2.2. RTL du full adder et de son testbench	9
2.2.3. Testbench du full adder comb	10
2.2.4. La simulation - Niveau 1 : commande minimale	10
2.3. Niveau 2 : Utiliser les filelists	11
2.3.1. Contrat logique du ripple carry	11
2.3.2. Les filelists	12
2.3.3. Utiliser la vue graphique	13
2.3.4. Ouverture de simulations précédentes	15
2.4. Niveau 3 : Wrapper réutilisable	16
3. Etape 2 : Synthèse logique - Outil Genus	18
3.1. Entrées et sorties	18
3.2. Niveau 0 : L'appel direct → mauvaise idée	20
3.3. Niveau 1 : Commande et Tcl minimale	21
3.3.1. Comment le script fonctionne	22
3.4. Comprendre les rapports	25
3.4.0.1. Fichier : report_timing.rpt - le timing	26
3.4.0.2. Fichier : report_power.rpt	27
3.4.0.3. Fichier : report_area.rpt et report_qor.rpt - Aire du design	27
3.5. Niveau 2 : script plus élaboré	28
4. Etape 3 : Implémentation physique - Innovus	29
4.1. Entrées et sorties	30
4.1.1. Les MMMC	30
4.1.2. Le format Open Access (OA) VS LEF	31
4.2. Niveau 1 : script minimal	32
4.3. Notions importantes à connaître	33

4.3.1. Niveau de métaux utilisés	33
4.3.2. skew	33
4.4. Ouverture du design dans Virtuoso	34
5. Etape 4 : Simulation gate-level	38
5.1. Attention au testbench	38
5.2. Comment savoir si le fichier sdf est bien pris en compte dans la simulation ?	39

1. Introduction

Ce tutoriel s'adresse à un public initié un minimum à l'aise avec linux (étudiant ingénieur ou ingénieur en microélectronique analogique). Il permet une prise en main **concrète** et **rapide** du flow numérique. Ce tutoriel n'a pas pour ambition d'être un guide ultime pour tout comprendre. Pour davantage de détails, il existe de très bonnes ressources sur internet. Parmi elles, la formation "RTL to GDSII" de Cadence ou une très bonne série de vidéo réalisée par Adi Teman dont vous pouvez retrouver le lien [ici](#) ou le github [ici](#).

Note : Dans ce parcours les encadrés en bleu sont généralement des notes ou des conseils alors que les encadrés en rouge font référence à des points importants et jalons de progression à ne pas manquer.

Le flow numérique (et a fortiori ce tutoriel) se découpe en trois grandes étapes dont chacune est prise en charge par un outil spécifique *:

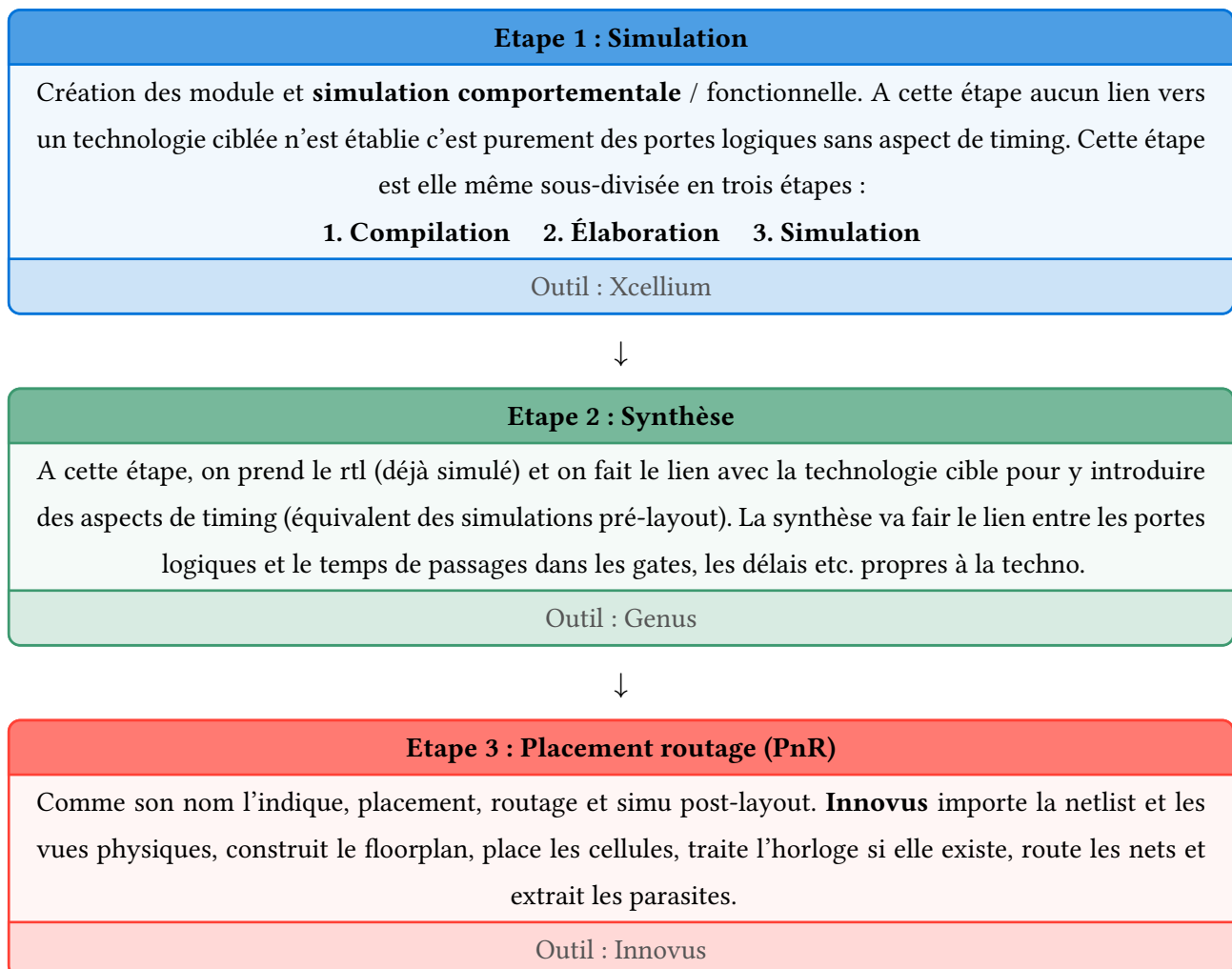


Figure 1: Méthodologie du flow numérique

Note :

Chacune de ces étapes est **essentielle**:

- La simulation répond à des questions fonctionnelles
- Le STA répond à des questions temporelles
- Les vérifications physiques répondent aux questions de géométrie et de connectivité.

Ces preuves de fonctionnement sont **complémentaires**.

* Il existe d'autres outils que ceux cités mais ils ne seront pas évoqués dans ce tutoriel. La démarche restant exactement la même d'un outil à l'autre, il faudra simplement réadapter les scripts en cas de changement d'outil.

1.1. Objectif

Pour illustrer les notions vues dans ce tutoriel, nous allons passer ensemble le flow numérique complet (du rtl jusqu'au layout) sur un bloc simple : un additionneur ripple carry 4 bits.

L'objectif de ce bloc est de faire une simple addition de 4 bits entre deux front d'horloge. Pourquoi les front d'horloge ? Car ceci permet de prendre en main une structure essentielle du flow numérique : un bloc combinatoire (on parle de bloc **combinatoire** quand des opérations sont faites et qu'il n'y a aucune horloge) entre deux **flip flop** (ie. bascules) qui font entrer les données sur un front d'horloge et lisent le résultat au front suivant comme illustré sur la **Figure 2**:



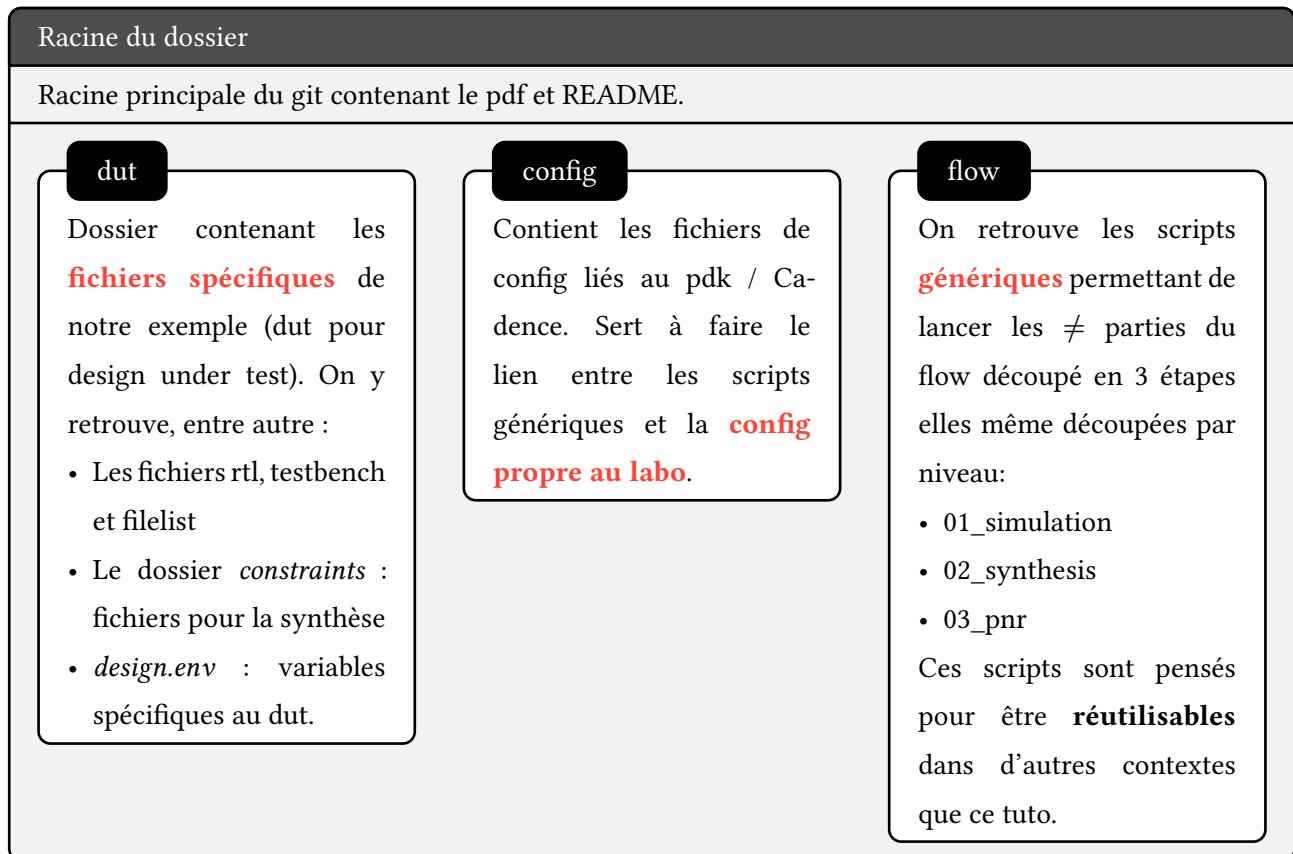
Figure 2: Schema de principe du circuit

Ce circuit permet, sans rajouter beaucoup de complexité, d'étudier des problématiques temporelles comme la latence, setup, hold, MMMC, CTS etc...

Méthodologie : Ne pas mettre la charrue avant les boeufs

Quand on fait quelque chose de nouveau (a fortiori en microélectronique numérique) la règle d'or est toujours de commencer simple et petit. Commencer petit permet de distinguer une erreur de **méthode** en s'affranchissant des difficultés réelles du design.

1.2. Arborescence du dossier du tutoriel



Arborescence

En microélectronique numérique il est nécessaire d'être **très au clair** avec l'arborescence des fichiers et des dossier. Il est important de définir **dès le départ** une structure claire.

Pour des raisons de répétabilité et de modularité, les scripts ne seront jamais écrits avec des PATH en dur mais seront toujours écrits en utilisant des PATH **absolus** chargés à partir de fichiers spécifiques.

La logique ici est que tous les fichiers spécifiques au dut soient dans le dossier *dut* tandis que les scripts dans *flow* sont **génériques** et utilisent des PATH ou variables définies dans *config* et *dut*.

La logique ici est de **toujours lancer les scripts dans le dossier "workdir" de chaque partie**. Il y a d'autres façons de procéder mais c'est la plus courante.

1.3. Approche pédagogique : méthode en plusieurs niveaux d'abstraction

Chaque chapitre montre d'abord:

1. L'appel direct à l'outil (quand c'est possible) avec une **commande minimale**. C'est le coeur de l'utilisation de l'outil et c'est ce qui sera appelé par les scripts dans la suite donc il est très important de bien maîtriser cette étape.

2. Dans un second temps, on utilise un script (wrapper) pour effectuer des vérifications, lancer la commande minimale, choisir un dossier de résultats et donner le résultat de manière structurée. En pratique, on utilise uniquement ce genre de script en microélectronique numérique car ils permettent de **gagner du temps**, d'**éviter les erreurs**, de **contrôler l'environnement de travail** et sont **réutilisables** et permettent donc à quelqu'un d'autre de l'utiliser et de détecter une potentielle erreur dans notre code.

Méthodologie : Ne pas automatiser une erreur

Un script plus long ne corrige pas une mauvaise filelist, un top incorrect ou un SDC incomplet.

→ **La commande minimale doit être comprise et validée avant d'ajouter l'automatisation.**

1.4. Informations pratiques

Disclaimer: Les commandes Cadence sont fondées sur les références installées¹²³. Leur disponibilité et leur syntaxe exacte doivent être confirmées dans l'environnement du laboratoire

Tout au long du tutoriel, on considère que l'environnement Cadence est chargé et que les commandes de lancement des outils xrun, genus, innovus sont chargées

Disclaimer 2 : La microélectronique numérique est une discipline qui fourmille de détails et de cas particuliers. On pourrait écrire 300 pages d'explications simplement sur la synthèse. Ce n'est pas l'objectif ici. Ce tutoriel permet de voir **en surface** les étapes du flow numériques. Il permet de comprendre comment les choses s'imbriquent entre elles et il constitue un bon point de départ pour comprendre le flow numérique dans sa globalité mais il **n'est pas suffisant** pour former un bon numéricien. Il est nécessaire par la suite d'approfondir les notions.

1. Xcelium Logic Simulation — Command-Line and Tcl Reference. Consulter la version installée et autorisée du laboratoire. Cadence Design Systems.
2. Genus Synthesis Solution — User Guide and Command Reference. Consulter la version installée et autorisée du laboratoire. Cadence Design Systems
3. Innovus Implementation System — User Guide and Command Reference. Consulter la version installée et autorisée du laboratoire. Cadence Design Systems.

2. Etape 1 : Simulation RTL - outil Xcelium

Cette première étape du flow se décompose elle même en trois phases (cf. **Figure 1**):

1. **Compilation** : Lecture du SystemVerilog et contrôle de syntaxe.
2. **Élaboration** : Construction de la hiérarchie et résolution des modules, paramètres et ports.
3. **Simulation** : Exécution des blocs temporels et production du verdict du testbench.

Dans la pratique, ces trois phases seront effectuées en **même temps** pas Xcellium.

Note : Erreurs courantes de la simulation

Généralement, une **erreur de syntaxe** appartient à la **compilation**. Un **top inconnu** ou un port impossible appartient à l'**élaboration**. Un \$fatal, un timeout ou une comparaison incorrecte appartient à la **simulation**.

2.1. Entrées et sorties

Voici les entrées sorties attendues pour cette phase de simulation :

Élément	Exemple	Rôle
RTL	full_adder_comb.sv	Description logique synthétisable
Filelist	rtl.f	Ordre et chemins des sources
Rapport	.sh ou .log	Rapport de sortie de la simulation (balises PASS ou FAIL)

Table 1: Entrées sorties de la phase de simulation (**entrées**, **sorties**)

On peut noter qu’il n’y a pas de sortie à proprement parler pour la simulation car la simulation ne produit pas de fichier, elle sert simplement à vérifier le bon fonctionnement de notre module. Notre “sortie” est donc le verdict des scripts.

2.2. Niveau 1 : Commande Xcelium minimale

Fichier : dut/rtl/full_adder_comb.sv

Dossier d’exécution : flow/01_simulation/workdir

Dans ce niveau, nous nous intéressons au module full_adder_comb. Module très simple et purement combinatoire (sans clock). La première étape consiste à savoir ce qu’on veut que le module fasse. Cette étape passe souvent par écrire une **machine à état** ou une **table de vérité** pour être au clair sur les fonctions **précises** du module.

2.2.1. Définition du contrat logique

Pour les entrées a, b et c_in :

- $\text{sum} = a \text{ XOR } b \text{ XOR } c_{\text{in}}$
- $c_{\text{out}} = ab + ac_{\text{in}} + bc_{\text{in}}$

a	b	$c_{\{\in\}}$	sum	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 2: Table de vérité du full adder combinatoire

2.2.2. RTL du full adder et de son testbench

Deuxième étape : on écrit le comportement du module en RTL (verilog, systemverilog ou vhd). Ici, cela donne :

```

1  `timescale 1ns/1ps -----> Définition d'un pas de simulation*
2
3  // Additionneur complet combinatoire sur un bit.
4  //
5  // Les equations sont ecrites explicitement afin que le lien entre la table
6  // de verite, le RTL et les portes obtenues apres synthese reste visible.
7  module full_adder_comb (
8      input  logic a_i,
9      input  logic b_i,
10     input  logic cin_i,
11     output logic sum_o,
12     output logic cout_o
13 );
14
15     assign sum_o = a_i ^ b_i ^ cin_i;
16     assign cout_o = (a_i & b_i) | (a_i & cin_i) | (b_i & cin_i);
17
18 endmodule

```

full_adder_comb.sv

Annotations dans l'image :

- Une boîte bleue sur la ligne 1 pointe vers le commentaire "Définition d'un pas de simulation*".
- Une boîte rouge sur la ligne 7-13 pointe vers le commentaire "Déclaration du module avec nom + Entrées/sorties".
- Une boîte verte sur la ligne 15-16 pointe vers le commentaire "Ce que fait le module".

* Les deux valeurs correspondent à:

- 1ns = unité de temps (si on met un délais de "10" dans le module cela signifiera alors 10 x 1ns)
- 1ps = précision temporelle / pas du simulateur (le simulateur peut distinguer les événements avec une résolution de 1 ps)

Note : Le DUT ne contient aucun délai, bloc initial ou appel système car c'est un bloc asynchrone (ie. purement combinatoire).

2.2.3. Testbench du full adder comb

Fichier : dut/rtl/tb_full_adder_comb.sv

De la même manière que un module, un testbench est aussi un module RTL qui comprend le module a tester ainsi que les entrée sorties commandées permettant de faire le test (exactement comme en analogique). Le rôle du testbench est également de faire ressortir des **marqueurs** (**PASS** ou **FAIL**) pour vérifier le bon fonctionnement comportemental de note module / DUT.

Note : Retrouver les codes sources

Pour des raisons de lisibilité, les codes abordés dans ce tutoriel se situent en annexe de ce document.

2.2.4. La simulation - Niveau 1 : commande minimale

Une fois le RTL du full adder et de son testbench écrits (et qu'on a donc une description comportementale de nos modules), il faut effectuer la simulation. Pour ce faire on execute (après avoir chargé Cadence et les outils) **depuis le dossier "01_simulation_minimal" de la simulation**:

```
xrun -64bit -sv -timescale 1ns/1ps \  
  ../../dut/rtl/full_adder_comb.sv \  
  ../../dut/rtl/tbfull_adder_comb.sv \  
  -top tb_full_adder_comb
```

→ Commande bash

Explication de la commande :

- **-64bit** : Utilise l'exécutable 64 bits.
- **-sv** : Active SystemVerilog (normalement on peut s'en passer car Xcellium est sensé reconnaître l'extension .sv mais cf note plus bas).
- **-timescale** : Fixe l'unité et la précision par défaut si un module n'a rien de spécifié (sinon utilisation du timescale du module).
- ***.sv** : Module ou sous module a simuler.
- **-top** : Choisit le top élaboré (ie. le module le plus haut dans la hiérarchie).

Note : Spécifications de commandes

En règle générale, il vaut mieux éviter de penser que "l'outil va comprendre tout seul". Effectivement, certaines choses sont prisent en compte automatiquement et on a vite fait de se dire qu'on ne va pas rajouter l'option "x" car ça alourdi inutilement. C'est **une erreur**. Plus on spécifie de chose, plus on est certain que ça fait ce qu'on veut. Cette approche évite de chercher (longtemps) une erreur bête alors que c'est simplement un problème d'appel mal compris pas l'outil (valable pour tous les outils du flow).

On s'attend a voir dans la sortie quelque chose comme:

```
xcelium> run  
TEST_PASS: full_adder_comb
```

[→Bash output](#)

Super, notre module est fonctionnel !

On peut voir en tapant "**ls**" que trois choses sont apparues :

- **Dossier xcelium.d** : Base de compilation et d'élaboration de Xcelium. Elle contient notamment le snapshot simulable et les données internes utilisées par Xrun.
- **xrun.log** : Journal texte de la compilation, de l'élaboration et de la simulation.
- **xrun.history** : Historique interne des invocations Xrun, utilisé par les fonctions d'historique et de rejou de commandes.

Dans notre cas nous n'en avons plus besoin on peut donc les supprimer:

```
rm -rf x*
```

[→ Commande bash](#)

→ Le full adder purement combinatoire fonctionne. On va pouvoir l'utiliser dans notre module de full adder pipeliné! Passons au niveau 2.

Note : Architecture de fichier

Il est important de bien avoir en tête d'où on lance le script et donc d'où partent les chemins relatifs!!! Chaque designer a son architecture de fichier et ses habitudes. La méthode ici consiste à toujours lancer la commande depuis la **racine du projet** et à adapter les chemins relatifs à ce point de départ plutôt que l'inverse.

2.3. Niveau 2 : Utiliser les filelists

Pour ce niveau et **dans toute la suite du tutoriel**, nous nous intéresserons au module ripple_carry_4. Autrement dit, à partir de maintenant nous laissons de côté la sous-brique full_adder_comb pour ne s'intéresser qu'au ripple_carry_4.

2.3.1. Contrat logique du ripple carry

Dans un premier temps, il est important de comprendre ce que fait notre module:

RIPPLE CARRY ADDER 4 BITS – CONTRAT LOGIQUE

1. A QUOI IL SERT ?

Le ripple carry adder 4 bits est un **additionneur binaire non signé** (ou signé en complément à deux si utilisé ainsi) qui additionne deux opérandes de 4 bits.

Il calcule la somme $S = A + B + Cin$.

Il est composé de 4 additionneurs complets (full adders) chaînés : la retenue (carry) de chaque étage se propage ("ripple") vers l'étage suivant.

2. INTERFACE LOGIQUE

• Entrées

- A[3:0] : premier opérande (4 bits)
- B[3:0] : second opérande (4 bits)
- Cin : retenue d'entrée (carry in)

• Sorties

- S[3:0] : somme (4 bits)
- Cout : retenue de sortie (carry out)

3. SCHÉMA BLOC



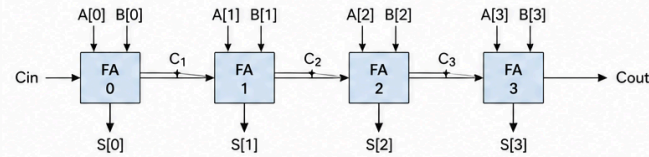
4. FONCTIONNEMENT

Le ripple carry adder 4 bits est constitué de 4 full adders en cascade.

À chaque étage i (de 0 à 3) :

- les bits $A[i]$ et $B[i]$ et la retenue entrante C_i sont additionnés
- on obtient la somme S_i et la retenue sortante C_{i+1}

La retenue se propage (ripple) du bit de poids faible (LSB, $i=0$) vers le bit de poids fort (MSB, $i=3$).



$$\text{Équation globale : } \{Cout, S[3:0]\} = A[3:0] + B[3:0] + Cin$$

5. EXEMPLE

A[3:0]	B[3:0]	Cin	S[3:0]	Cout	Interprétation
0101 (5)	0011 (3)	0	1000 (8)	0	$5 + 3 + 0 = 8$
1111 (15)	0001 (1)	0	0000 (0)	1	$15 + 1 + 0 = 16 \rightarrow 0000$ et $Cout=1$
1001 (9)	0110 (6)	1	0000 (0)	1	$9 + 6 + 1 = 16 \rightarrow 0000$ et $Cout=1$
0000 (0)	0000 (0)	0	0000 (0)	0	$0 + 0 + 0 = 0$

6. CARACTÉRISTIQUES

- Largeur : 4 bits
- Type : additionneur à propagation de retenue (ripple carry)
- Latence logique : proportionnelle au nombre de bits (la retenue doit traverser jusqu'à 4 étages)
- Utilisation typique : addition simple, ALU basique, compteurs, calculs d'adresses, etc.

Figure 3: Contrat logique du ripple_carry_4 [généré par ChatGPT]

2.3.2. Les filelists

Comme nous pouvons le voir sur la **Figure 3**, le module ripple_carry_4 contient le sous-module full_adder_comb (en plusieurs instances). Pour la simulation, on peut exécuter xrun individuellement pour chaque DUT et chaque testbench (comme avec le niveau 1) mais une bonne pratique si on a beaucoup de fichiers RTL est d'utiliser une filelist (extension en .f) pour définir **dans quel ordre** procéder à l'élaboration et qu'on ait pas de problèmes de dépendances non résolues à cause d'un mauvais ordre. Par ailleurs, ces filelists seront **nécessaires pour les étapes suivantes** donc autant les implémenter dès maintenant.

Note : Utilisation des filelists

L'utilisation des filelists n'est pas **un passage obligatoire**. Certaines personnes utilisent des scripts ou il appellent chaque module séparément comme dans le niveau 1.

À retenir : Séparer les rtl des tb

Il est **primordial** de séparer les .f des modules DUT (dans le dossier **rtl**) et des testbench en deux fichiers distincts. En effet, les filelists rtl.f seront **réutilisées** par la suite dans les phases de synthèse et de pnr et ne servent pas seulement pour la simulation.

C'est le moment de parler d'une notion importante : Dans la suite du tutoriel tous les chemins seront indiqués

relativement à la racine du tutoriel. Pour ce faire on va définir une variable d'environnement qui va nous permettre de spécifier le chemin **absolu** de la racine du tutoriel :

```
export DESIGN_PATH=/chemin/vers/la/racine/du/dossier
```

→ Commande bash

Maintenant, à chaque fois qu'on va appeler \$DESIGN_PATH le code va pouvoir retrouver le chemin depuis n'importe quel répertoire d'exécution (car chemin absolu).

Voici, par exemple, ce que contient la filelist rtl.f :

```
1 $DESIGN_PATH/dut/rtl/full_adder_comb.sv
2 $DESIGN_PATH/dut/rtl/ripple_carry_4.sv
```

txt

rtl.f

C'est simplement une liste de tous les modules à élaborer et simuler. Une fois les filelists écrites on peut lancer (toujours depuis le dossier "01_simulation_minimal"):

```
xrun -64bit \  
-sv \  
-access +rwc \  
-gui \  
-timescale 1ns/1ps \  
-f $DESIGN_PATH/dut/filelists/rtl.f \  
-f $DESIGN_PATH/dut/filelists/tb.f \  
-top tb_ripple_carry_4  
-mess
```

→ Commande bash

Explication de la commande :

- **-access +rwc** : Donne les droits de prober tous les signaux dans la hiérarchie du design.
- **-gui** : Ouvre la vue graphique
- **-f** : Lit une filelist.
- **-mess** : Affiche tous les messages en détail

2.3.3. Utiliser la vue graphique

Cette fois, avec l'option **-gui**, la vue graphique s'ouvre et on peut visualiser les résultats de simulation:

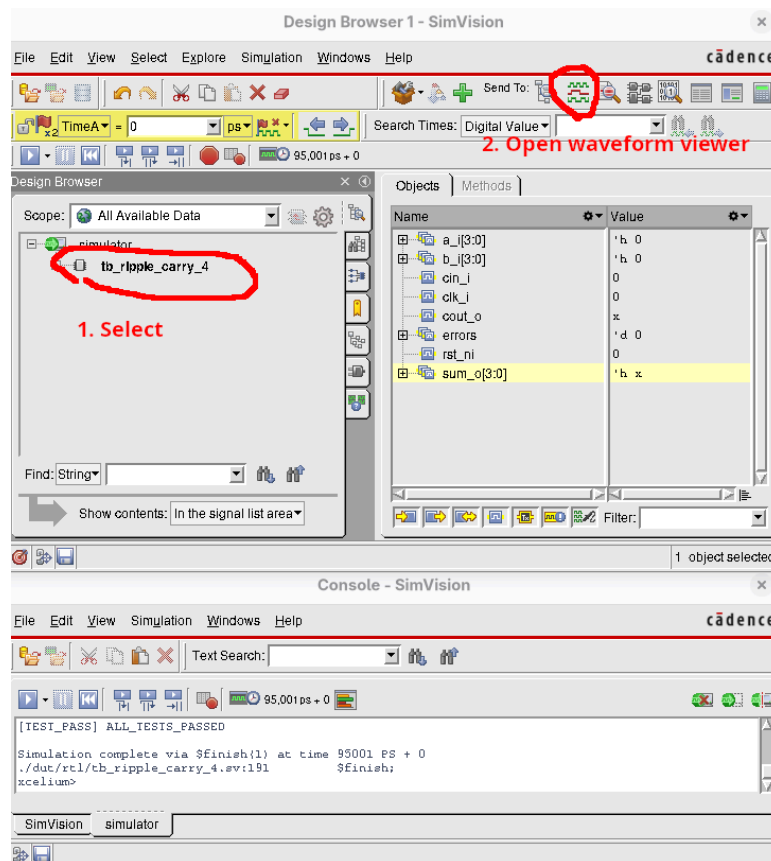


Figure 4: Simvision fenêtre 1

Après ouverture de la waveform cette fenêtre apparaît:

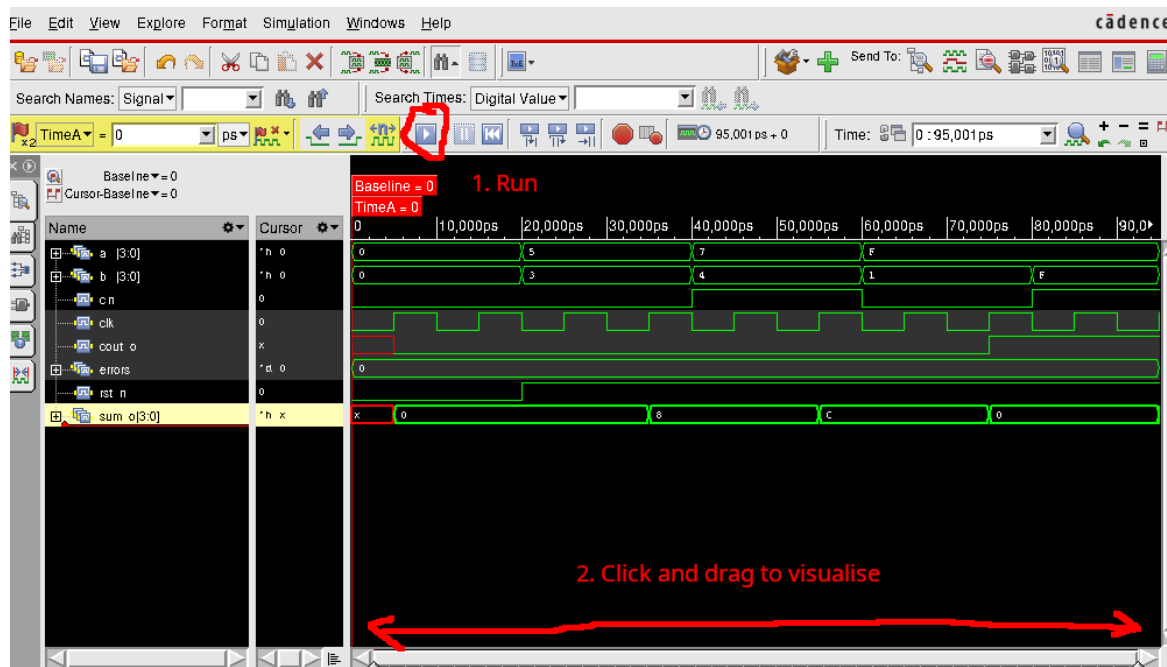


Figure 5: Simvision fenêtre 1

Si d'aventure nous avons des problèmes de simulation, cette vue graphique est un outils essentiel pour comprendre pourquoi tel ou tel signaux n'est pas dans l'état attendu.

Note : Utilisation de l'interface graphique

Dans l'idée, on évite d'utiliser l'interface graphique et on ne l'ouvre **que pour du debugage** si le testbench revoie **FAILED**.

2.3.4. Ouverture de simulations précédentes

On peut remarquer que cette fois, en plus de ce que xcellium a généré au niveau 1, on a un dossier **waves.shm** qui s'est créé. Ce dossier contient toutes les waveforms (les signaux temporels) qui ont été créés. C'est en réalité ces signaux qu'on a affichés en **Figure 5** et il est possible de les visualiser après coup en tapant :

```
simvision waves.shm
```

→ Commande bash

Une fenêtre va s'ouvrir et il est alors possible de consulter les signaux. Une autre chose intéressante à consulter est la **netlist générée**. La consulter permet de vérifier que ce que nous avons en tête est bien conforme à ce qui va être synthétisé par la suite. Pour l'ouvrir :

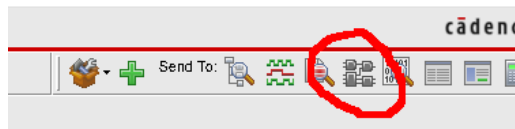


Figure 6: Simvision : ouverture de la netlist

Puis une fenêtre s'ouvre et en double-clickant sur les modules on peut rentrer dedans. On peut alors visionner nos différents modules :

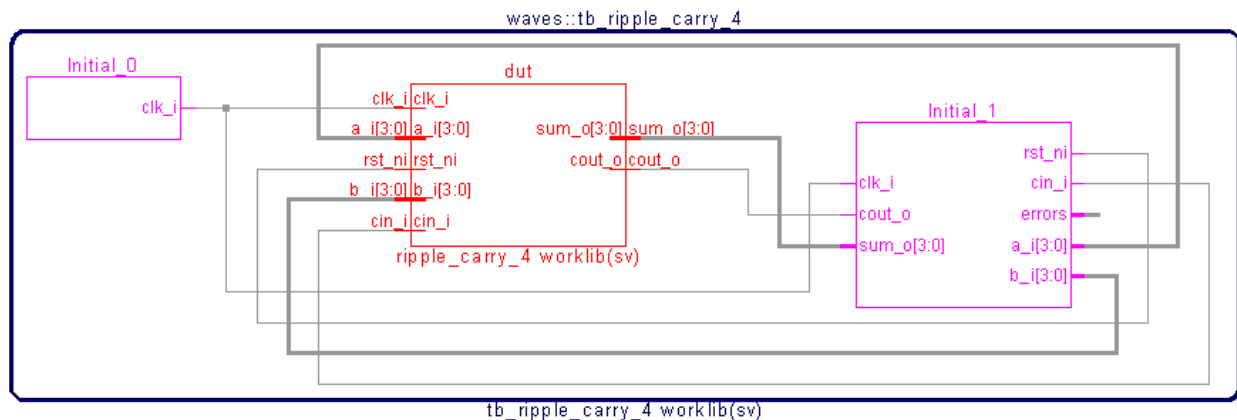


Figure 7: Simvision : Module du tesbench

La **Figure 7** illustre bien le fait que notre testbench est un module qui **encapsule le dut** et qui comprend d'autres modules pour gérer les entrées et sorties attendues. On peut rentrer dans le dut et cela donne :

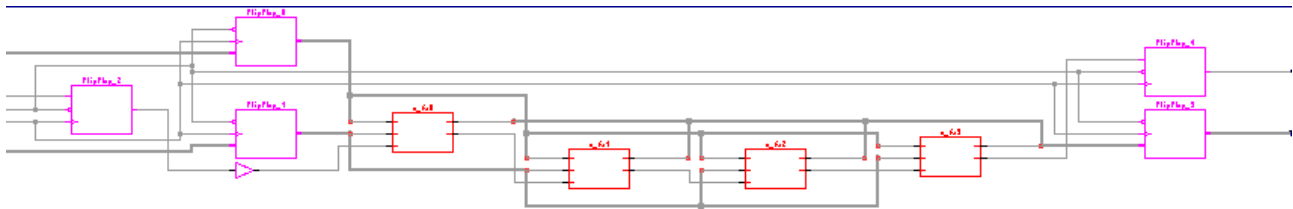


Figure 8: Simvision : Module ripple_carry

Sur la **Figure 8** on voit bien les full adder combinatoires (en **rouge**) dont les entrées/sorties sont clockées par des bascules D (en **violet**). Enfin, on peut également visualiser l'intérieur de nos full adder composés de portes logiques:

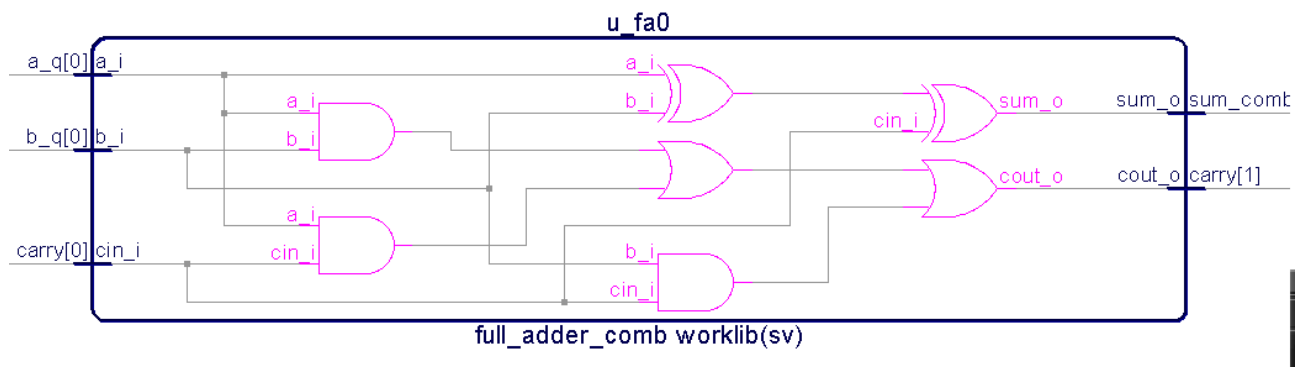


Figure 9: Simvision : Module full adder

De la même manière, Xcelium va mettre ses fichiers dans le répertoire d'exécution, si il n'y a pas d'erreurs, nous n'en avons plus besoin:

```
rm -rf x* waves.shm
```

→ Commande bash

2.4. Niveau 3 : Wrapper réutilisable

Dossier : flow/01_simulation/02_simulation_gate_level

Comme déjà mentionné précédemment, en pratique, on ne va pas lancer les commandes à la main à chaque fois car:

1. Ça peut être source d'erreur.
2. Il nous faut avoir un flow répétable et qu'on puisse facilement relancer.

C'est pourquoi on va privilégier l'utilisation d'un wrapper qui va tout automatiser pour nous.

La encore, il y a autant de façon de faire que de designer mais nous allons ici voir la méthode la plus couramment utilisée : le makefile.

Nous n'allons pas revenir ici sur ce qu'est un makefile ni comment l'utiliser mais ce qu'il fait comprendre c'est qu'il permet de lancer une suite d'instruction en utilisant la balise associée. Ici par exemple, on peut lancer la simulation avec environnement graphique en faisant :

```
make run_sim
```

[→ Commande bash](#)

Attention : Il faut bien avoir en tête qu'il le makefile utilisé est relativement simple et qu'il faut bien veiller à exporter les PATH corrects et pointer vers les bons fichiers RTL **dans le bon ordre** pour que ça fonctionne.

Ensuite, une fois la simulation finie et analysée, nous pouvons nettoyer le répertoire de run avec :

```
make clean
```

[→ Commande bash](#)

et relancer en vue non graphique avec :

```
make run_sim_no_gui
```

[→ Commande bash](#)

Note : Utilisation de script shell (.sh)

Une autre méthode couramment utilisée pour automatiser la simulation est l'utilisation de scripts shell plus ou moins complexes permettant de mettre à jours les dut facilement, ranger des résultats dans des dossiers séparés, Vérifier si tous les tesbench sont fonctionnels ect..

Ici, nous préférons utiliser le makefile car beaucoup plus simple a prendre en main et beaucoup plus proche de l'utilisation de la commande directe.

Attention : Export dans un script

Si on fait "export PATH_CUSTOM=/le/chemin" dans un programme shell (ex: run_sim.sh) et qu'on l'exectute avec la commande **bash** la variable PATH_CUSTOM sera détruite à la fin de l'exécution du programme. Pour éviter le l'exporter à chaque fois il vaut mieux faire l'export dans le terminale directement plutôt que dans le script.

Nous allons revenir sur la simulation gate level dans la **Section 5**.

Maintenant que nous avons simulés notre DUT, nous pouvons passer à synthèse.

3. Etape 2 : Synthèse logique - Outil Genus

La deuxième étape de ce parcours du combattant est donc la synthèse. C'est à cette étape que le lien avec la technologie cible va s'effectuer. Cette étape est plus complexe que la précédente car il va falloir correctement établir le lien avec le pdk et qu'il y a davantage de paramètres à prendre en compte.

En pratique, Genus transforme un **RTL** en une **netlist composée de cellules de la bibliothèque cible**. Il optimise la logique sous les contraintes de :

- Timing
- Aire
- Transition
- Fanout
- Charge

Comme pour la simulation, l'outil Genus va réaliser plusieurs étapes d'un seul coup parmi elles on peut citer les plus importantes :

Etape	Description
1	Lire le RTL, l'élaborer et comprendre le SDC
2	Exécuter syn_generic : transforme le RTL en logique générique et simplifie les expressions
3	Exécuter syn_map : choisit des cellules disponibles dans la Liberty
4	Exécuter syn_opt : améliore timing, aire et règles électriques selon l'effort demandé
5	Générer les rapports et les deux fichiers nécessaires à Innovus

Table 3: Etapes clés réalisées lors de la synthèse

3.1. Entrées et sorties

Dans un premier temps il est primordial de comprendre ce dont on a besoin pour lancer la synthèse et quels sont les fichiers en sortie de synthèse :

Élément	Exemple	Rôle
RTL	ripple_carry_4.sv	Description logique synthétisable
Filelist	rtl.f	Ordre et chemins des sources
Contraintes	constraints.sdc	Clocks, délais d'E/S et exceptions
Liberty	stdcells_tc.lib	Fonctions, arcs, délais et puissance
Environnement	conf_ihp130.env	Chemin absolu des fichiers
Netlist mappée	ripple_carry_4_netlist.v	Netlist avec cellules du PDK
SDC exporté	ripple_carry_4_sdc.sdc	Contraintes de timing mappées
Fichiers SDF	ripple_carry_4_delay.sdf	Extraction de parasites
Rapports	report_timing.rpt, report_area.rpt, report_qor.rpt	Preuves à examiner

Table 4: **entrées**, **sorties** de la minimales de la synthèse logique

Note importante : Il n'y a QUE des cas particuliers

Les entrées sorties présentées ici permettent d'aller au bout du flow mais il en existe d'autres! Il est donc possible de rajouter des entrées/sorties supplémentaires comme les lef en entrée, mmmc en sortie ect. dépendamment du type de flow, du pdk, de l'outil etc...

Il y a autant de manière de faire que de designer / outil / pdk etc.. Ici, ce qui est donc important c'est de **comprendre la philosophie**, les briques élémentaires qu'on retrouve dans tous les types de flow.

On ne va pas revenir sur ce que sont les fichiers .sv et .f mais voici une rapide description des autres entrées :

- **Fichiers .env (variables d'environnement)** : C'est un fichier de définition de variables que l'on va utiliser dans les différents scripts pour la synthèse et le PnR. Ces fichiers permettent d'écrire des scripts génériques utilisant des variables comme \$FICHIER_SDC qu'on va définir une bonne fois pour toute dans les .env plutôt que d'écrire le PATH à la main dans chaque fichier et devoir tout changer partout au moindre changement de tech ou de dut.
- **Fichier SDC (Synopsys Design Constraint)**: Le fichier qui va définir les contraintes de timing à respecter pour notre design. Genus a besoin du SDC pour savoir quelle fréquence il doit viser pendant la synthèse.

Note : Pourquoi un fichier .sdc en entrée et un autre en sortie ?

Le fichier .sdc en **entrée** indique Le budget disponible pour un chemin entrée-vers-sortie est approximativement : [**période - délai d'entrée - délai de sortie - marges internes**]. Il correspond à un objectif théorique mais n'est pas lié à la techno ciblée ni aux cellules utilisées. Il est uniquement dépendant du **cahier des charges**.

Le fichier .sdc de sortie donne le budget de temps **en utilisant les cellules du pdk**. Il va définir très précisément quelle est la deadline pour que telle ou telle cellule reçoive ou envoie son signal. C'est un fichier à fournir à Innovus pour le PnR afin qu'il sache après placement routage si le timing est toujours respecté.

- **Fichiers .lib (liberty)** : Ce sont les fichiers du PDK qui définissent les caractéristiques des cellules de la technologie cible. Ces fichiers répertorient les temps de passage dans chaque cellule. Il existe 3 types de liberty:
 - tc ou tt pour typical (ie. le temps moyen de passage dans la cellule)
 - wc ou ss pour worst case / slow slow (temps le plus lent)
 - bc ou ff pour best case / fast fast (temps le plus rapide)
- **Fichier SDF (Standard Delay Format)** : [optionnel] - Fichier pour faire des simulations après extractions de parasites. Après la synthèse ça n'a pas beaucoup d'intérêt car ce qui nous intéresse c'est l'extraction de parasites **après placement routage**.

3.2. Niveau 0 : L'appel direct → mauvaise idée

Comme il y a beaucoup d'étapes à effectuer lors de la synthèse, un appel direct à l'outil n'est pas recommandé car genus prend énormément de paramètres. Un exemple d'appel direct à genus pourrait être :

```
genus -no_gui \
  -execute "read_hdl -sv chemin/vers/ripple_carry_4.sv" \
  -execute "elaborate ripple_carry_4" \
  -execute "read_sdc chemin/vers/ripple_carry_4.sdc" \
  -execute "syn_generic" \
  -execute "syn_map" \
  -execute "syn_opt" \
  -execute "write_hdl -hier > chemin/vers/ripple_carry_4.mapped.v" \
  -execute "write_sdc > chemin/vers/ripple_carry_4.mapped.sdc" \
  -log genus.log
```

→ Commande bash

Note : Ce code est donné à titre d'exemple. L'appel direct à genus ne fonctionne pas en l'état (mauvais chemin etc.)

Explications de la commande :

On retrouve les grandes étapes répertoriées dans la **Table 4** :

1. **read_hdl** : compile les sources
2. **elaborate** : construit la hiérarchie et résout les paramètres
3. **read_sdc** : applique l'intention de timing
4. **syn_generic** : transforme le RTL en logique générique et simplifie les expressions
5. **syn_map** : choisit des cellules disponibles dans la Liberty
6. **syn_opt** : améliore timing, aire et règles électriques selon l'effort demandé
7. **Rapports** : qualifient le résultat
8. **Exports** : alimentent Innovus

Inconvénients de procéder ainsi:

- Chemins relatifs en dur → peu portable, source d'erreur.
- Pas de gestion d'erreur (si une étape échoue, Genus continue).
- Pas de rapports (pas de report_timing.rpt, etc.)
- Pas de répertoire de travail avec des sorties bien rangées.
- Et surtout **incomplet !**. Il manque beaucoup de choses et de paramètres pour que la synthèse aboutisse.

Conclusion : C'est une **mauvaise méthode**.

Note importante : Etapes de la synthèse

On peut voir sur l'appel direct les **étapes clés indispensables** pour réaliser la synthèse. Evidemment, beaucoup de paramètres supplémentaires sont à ajouter pour affiner la synthèse mais l'essentiel est là.

Ici, nous avons un DUT simple et donc nous pouvons **regrouper** les étapes de la synthèse en un seul fichier. Cependant, il est courant de **séparer** les différentes étapes de la synthèse (syn_generic, syn_map, syn_opt ect.) en **plusieurs fichiers** pour vérifier qu'ils marchent et les optimiser un par un.

3.3. Niveau 1 : Commande et Tcl minimale

Dossier : flow/02_synthesis/01_syn_minimal/

On vient de le voir, contrairement à la simulation, Genus nécessite d'être configuré pour correctement fonctionner. En effet, il y a davantage d'étapes : le lien avec le PDK, les différents fichiers .f, .sdc etc. et tout ceci nécessite des scripts pour être fait correctement et dans le bon ordre.

1ère étape : se mettre dans le dossier de travail et exporter le DESIGN_PATH:

```
cd flow/02_synthesis_genus/01_syn_minimal/workdir/
export DESIGN_PATH=/chemin/vers/la/racine/du/dossier
```

→ Commande bash

Le langage de script qui va nous permettre de configurer genus est le **.tcl**. C'est dans ce fichier qu'on va renseigner ce que genus doit faire et dans quel ordre. Pour lancer genus en utilisant le fichier .tcl de configuration la commande minimale est :

```
genus -legacy_ui \  
-file ../syn_minimal.tcl \  
-log ./logs/genus
```

[→ Commande bash](#)

Explication de la commande :

- **-legacy_ui** : Ouvre Genus en mode **legacy** (il reconnaît alors les commandes utilisées dans le script).
- **-file** : Permet de dire quel fichier de script utiliser.
- **-log** : Permet de spécifier où ranger ses fichiers de log et quel nom leur donner (non essentiel dans l'absolu mais recommandé).

Note :

Si la variable DESGN_PATH n'a pas été exportée, on devrait avoir une sortie proche de :

```
can't read "::env(DESIGN_PATH)": no such element in array  
Encountered problems processing file: flow/02_synthesis/01_syn_minimal/  
syn_minimal.tcl
```

[→ Bash output](#)

3.3.1. Comment le script fonctionne

Le script de la synthèse effectue 2 opérations principales :

1. Aller chercher les variables spécifiques au projet et au pdk
2. Exécuter ligne par ligne les étapes de la synthèse.

Voici un petit schéma des imbrications d'appels:

Terminal

Terminal d'où est lancée la commande `genus`. Les outils cadence sont sourcés (les commandes **xrun**, **genus**, **innovus** par exemple sont accessibles) ainsi que la variable **\$DESIGN_PATH** définie par la commande `export DESIGN_PATH=/chemin/vers/la/racine/du/dossier`

genus

Genus exécute **ligne par ligne** les commandes du script. Il commence par charger les variables d'environnement utilisés pour la simulation (ex : **\$GENUS_SDC**) qui sont définies dans les deux fichiers **.env**:

1. Le fichier qui définit toutes les variables liées à la techno utilisée : `$env(DESIGN_PATH)/config/conf_ihp130.lib`. On y retrouve par exemple :

```
set GENUS_LIBERTY_TC chemin/vers/le/typical.lib
set GENUS_LIBERTY_BC chemin/vers/le/best_case.lib
...
```

2. Le fichier de variables propre au dut : `$env(DESIGN_PATH)/dut/design.lib`. On y retrouve par exemple :

```
set GENUS_FILELIST chemin/vers/le/rtl.f           # spécifique au projet
set GENUS_SDC chemin/vers/le/constraint.sdc        # spécifique au projet
...
```

Ainsi, quand on veut changer de dut, il nous suffit simplement de pointer vers un autre `design.env`. De même pour changer de pdk on peut simplement changer de `config.env`.

Ensuite, il déroule le flow de synthèse ligne par ligne et s'arrête en cas d'erreur.

Figure 10: Imbrication des appels des fonctions de la synthèse

Comprendre le script tcl

Nous n'allons pas nous attarder énormément ici sur le contenu du script tcl car c'est une discipline à part entière, avec toutes ses complexités. Chaque version de Genus a ses propres commandes, ses subtilités et surtout **chaque design** va nécessiter la mise en place de paramètres très spécifiques.

La version minimale du script donne vraiment les commandes **obligatoires** et laisse l'outil en automatique. Dans la pratique ça n'est pas aussi simple ; tout le travail réside dans la subtilité.

Note : Utilisation de Genus ligne par ligne

Plutôt que de lancer Genus avec un script on aurait tout aussi bien pu le lancer avec la commande `genus -legacy_ui` et **exécuter les lignes du script** les unes à la suite des autres.

Méthode : Repérer les erreurs

Lorsque l'on lance des gros scripts, une erreur peut vite se glisser, se perdre et devenir irreparable (car non bloquant pour Genus par exemple). Pour prévenir ce problème il y a une règle de bonne pratique toute simple : **On découpe le flow en sous-étapes.**

Note : Execution de Genus

Il faut bien avoir en tête que le fichier .tcl n'est pas executé en tant que tel, c'est genus qui est executé et qui lance les commandes lignes par lignes. Ainsi, le script ne peut pas charger des variables d'environnement **globale** et une fois sorti de genus (avec la commande **exit**) les variables comme **\$GENUS_SDC** ne seront plus définies.

Une fois le script terminé, on peut ouvrir la vue graphique (normalement pas nécessaire à cette étape mais intéressant de voir ce qu'il a généré). Pour ce faire, dans genus :

```
legacy_genus:/> gui_show
```

→ Commande bash

Puis selectionner la vue *Schematic* :

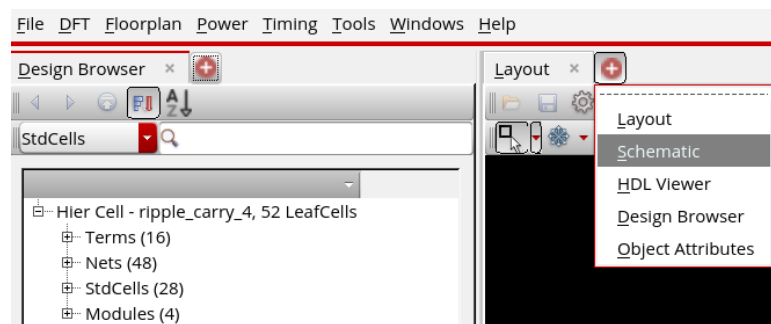


Figure 11: Genus : comment ouvrir la vue netlist

On peut alors observer la nouvelle netlist générée par genus avec cette fois-ci les noms des composants du pdk:

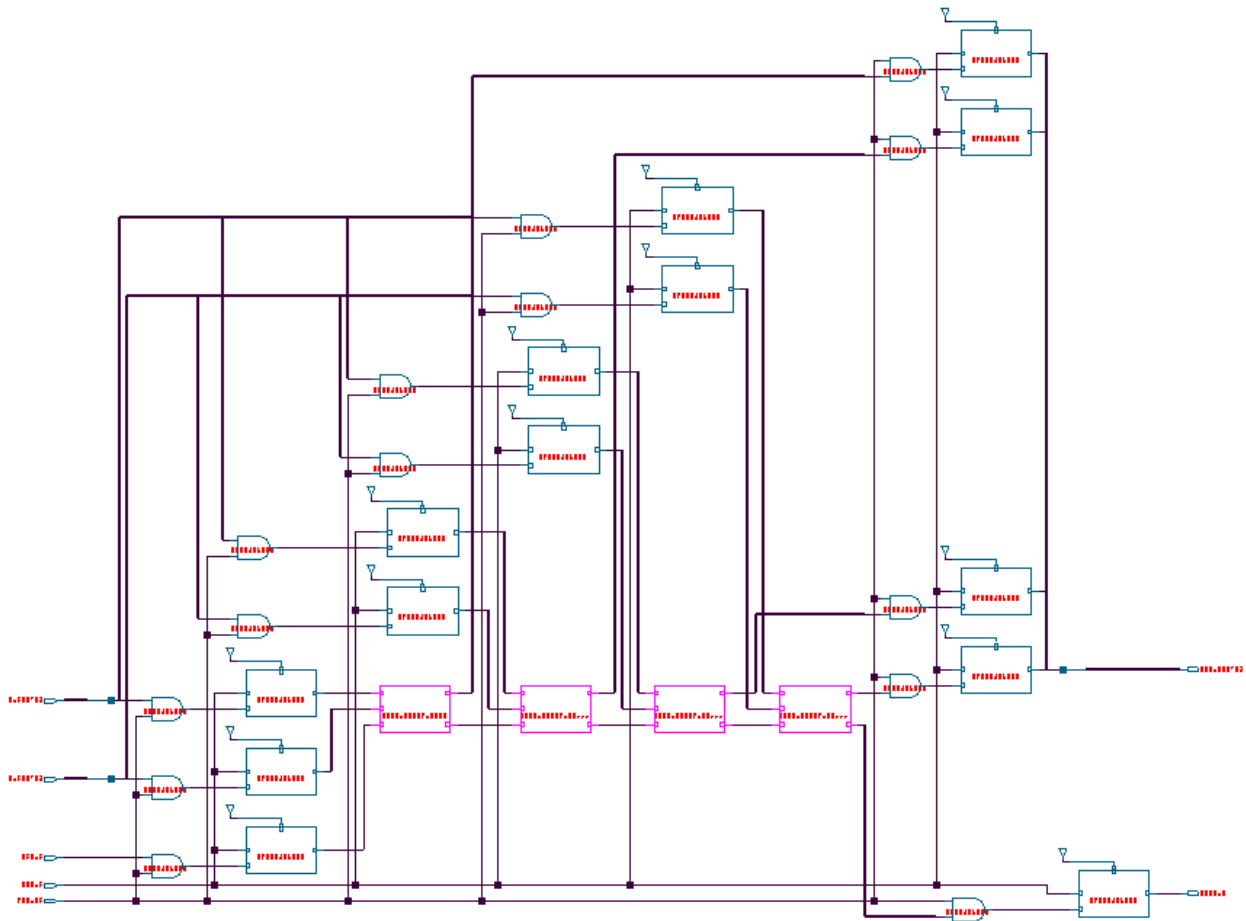


Figure 12: Netlist générée par Genus

Une fois qu'on en a terminé avec Genus, on peut le fermer :

```
legacy_genus:/> exit
```

→ Commande bash

3.4. Comprendre les rapports

Maintenant qu'on a fait la première synthèse, on peut regarder les résultats dans le répertoire d'où on a lancé Genus en tapant la commande :

tree

#ou

ls

→ Commande bash

On y trouve une arborescence de ce type :

- **fv/** (pour la vérification formelle, pas utilisé ici)
 - ↳ ripple_carry_4/
 - ↳ fv_map.fv.json
 - ↳ fv_map.map.do
 - ↳ fv_map.singlebit.original_name.alias.json.gz
 - ↳ fv_map.user_hier.json

- ↳ fv_map.v.gz
- ↳ read_libs.tcl
- ↳ rtl_to_fv_map.do

- **logs/**

- ↳ genus.cmd : la commande appelée
- ↳ genus.log : le log de Genus

- **outputs/** (les sorties utilisées par la suite dans le PnR)

- ↳ (ripple_carry_4_delays.sdf)
- ↳ ripple_carry_4_netlist.v
- ↳ ripple_carry_4_sdc.sdc

- **reports/**

- ↳ report_area.rpt
- ↳ report_power.rpt
- ↳ report_qor.rpt
- ↳ report_timing.rpt

Alors, qu'est ce qu'il faut regarder ?

La procédure typique est :

- Si on a une erreur → on regarde le log (ici **genus.log**) pour comprendre d'où vient l'erreur et la corriger.
- Si non, on regarde les **rapports** pour vérifier que le circuit correspond aux attentes (et, si oui, c'est terminé pour cette étape et on peut fournir les **outputs** à innovus pour le PnR).

Note : Erreur de synthèse types

Une erreur avant *elaborate* concerne souvent les fichiers ou la syntaxe. Une *référence non résolue* après élaboration concerne la hiérarchie. Un *timing mauvais* après mapping concerne plutôt contraintes, architecture ou choix de cellules.

Mais alors comment lire les rapports et à quoi correspondent ils ?

3.4.0.1. Fichier : report_timing.rpt - le timing

Comme son nom l'indique c'est dans ce fichier qu'on retrouve les informations relatives au timing. C'est ici que l'on voit si le timing respecte les contraintes définies dans le sdc.

Concept essentiel à comprendre à cette étape : la **slack**. Elle est définie comme suit :

$$\text{slack} = t_{\text{requis}} - t_{\text{arrivee}}$$

Une slack **négative** indique qu'on est en retard → **violation des contraintes**.

Pour vérifier, on peut parcourir le fichier à la main ou lancer une commande du type :

```
cd workdir # Si pas déjà fait
grep -i "slack" reports/report_timing.rpt
```

→ Commande bash

Dans cet exemple on peut voir qu'on a un seul path/chemin pour les données mais dans un circuit plus élaboré on va en avoir beaucoup. Dans ce cas le rapport sera trié du chemin avec la plus mauvaise slack à la meilleure (on peut également lui mettre en paramètres combien de path maximum il met dans le rapport).

Note : La slack

Le corner le plus intéressant à regarder pour la slack est le WC car c'est ce corner qui menera au temps de setup le plus lent.

GENUS PASS ≠ Design fonctionnel

Le code retour de l'outil, l'élaboration, les contraintes, le timing, les design rules et les exports sont des **contrôles distincts**, Ils permettent de savoir ou le flow à éventuellement planté. Un code de retour : **GENUS_STATUS=PASS** signifie **uniquement** que toutes les commandes ont été exécutées et les fichiers attendus ont été générés. Cela ne signifie **pas** nécessairement que :

- le slack est positif
- On a une absence de warnings dans check_design
- On a une absence de warnings dans check_timing_intent
- Une bonne fermeture setup/hold
- On a un résultat signoff

Les **rapports doivent donc toujours être examinés**, même lorsque la commande s'est terminée normalement.

On peut faire cette vérification à la main ou l'intégrer dans un script (cf. niveaux suivants).

3.4.0.2. Fichier : report_power.rpt

Indique la consommation hésitée du circuit. Il faut bien avoir en tête que la consommation finale et fiable du circuit sera donnée à la suite du PnR. Ici, la synthèse donne déjà une bonne héstimation qui est déjà sous-évaluée. Si la consommation est trop importante pour les spéc à cette étape, ça ne sert à rien d'aller plus loin et il faut retravailler la synthèse ou le RTL avant de continuer.

3.4.0.3. Fichier : report_area.rpt et report_qor.rpt - Aire du design

Rapport de la surface prise par notre circuit. Il faut notamment vérifier :

- **Aire totale** : nombre de cellules + répartition combinatoire/séquentielle
- **Cellules non mappées** : doit être à 0

Pareil que pour la consommation, si l'aire est trop grande à cette étape c'est bloquant → il faut retravailler le script de synthèse ou le design RTL.

Note : Aire vs timing

Une aire plus petite **n'est pas automatiquement meilleure** si elle dégrade le timing ou la robustesse électrique. Si, toutefois, on souhaite optimiser l'aire au détriment par exemple du timing on peut le spécifier à genus avec des commande du type :

```
1 set_db syn_goal {area 1.0 timing 0.8} # effort 100% aire, 80% timing
2 set_db syn_timing_slack_margin 0.05 # Marge de 50ps sur le slack
```

tcl

Genus essayera de réduire l'aire, mais ne sacrifiera pas le timing au-delà de la marge définie par syn_timing_slack_margin.

Ceci est un exemple de maitre de l'outil plus fin qu'on peut attendre dans un niveau 2 de synthèse plus avancé.

3.5. Niveau 2 : script plus élaboré

Le script du niveau 1 est fonctionnel sur le papier mais en pratique il est **incomplet**. Notre circuit étant très simple, Genus à réussi à faire un synthèse fonctionnelle (qui respecte les timings de constraints.sdc) tout seul. En règle générale, ça n'est pas le cas et le métier des designer numérique est justement de venir jouer sur les paramètres de genus pour orienter son travail et aboutir à une synthèse fonctionnelle. Ce travail est en général très spécifique à un projet (chaque projet ayant ses propres contraintes) et il n'y a pas une manière de faire.

Ici, aller plus en détail sur le script de synthèse n'a pas beaucoup d'intérêt car pour spécifier davantage de choses à Genus il faudrait savoir **précisément ou on veut aller et ce qui ne va pas**.

Dans l'idée, quand la synthèse devient plus complexe, on va séparer le flow en plusieurs scripts et les lancer l'un à la suite de l'autre. De même que pour l'implémentation, il n'est pas très pertinent ici de mettre un exemple de flow plus maitrisé car les commandes et les outils peuvent vite évoluer. Ce qui est important c'est de bien comprendre **quelles sont les étapes importantes de la synthèse** et donc qu'est ce que je peux modifier / sur quoi je peux jouer.

4. Etape 3 : Implémentation physique - Innovus

Pour cette phase, qu'on appelle indifféremment implémentation physique ou placement routage (place and route (PnR) en anglais), nous allons utiliser l'outil Innovus. Comme pour les autres logiciels, Innovus possède plusieurs version ayant chacune leur syntaxe. Nous allons utiliser Innovus avec la syntaxes **stylus** grâce à la commande :

```
innovus -stylus
```

→ Commande bash

Note : Stylus

Stylus est un outils développé dans les dernières versions des outils par Cadence dans le but **d'unifier les phases de synthèse et de PnR**. L'objectif affiché est de permettre d'utiliser ce seul outil pour dérouler tout le flow. Ici, nous allons utiliser Innovus avec la syntaxe stylus mais il est important de comprendre qu'on ne va effectuer que le PnR avec Innovus (la synthèse ayant été réalisée avec Genus)

Comme pour la synthèse, l'outil Innovus va réaliser plusieurs étapes d'un seul coup parmi elles ont peut citer les plus importantes :

Etape	Description
1	Lire le fichier MMMC avec read_mmmc
2	Lire les fichiers du pdk décrivant le layout des standard cell avec read_physical
3	Lire la netlist générée par Genus avec read_netlist
4	Initialiser sa database interne avec init_design et définir un certain nombre de paramètres avec set_db notamment
5	Générer le floorplan (<i>ie.</i> quelle est la taille de la puce, ect.) avec create_floorplan
6	Générer les pin d'entrée/sorties (IO) avec assign_io_pins
7	Générer les éléments liées à l'alimentation (power ring, stripes)
8	Faire le placement (<i>ie.</i> ou sont les cellules) avec place_design
9	Générer le clock tree (CTS) (<i>ie.</i> comment est routé le signal de clock) avec cchopt_design
10	Router le design (<i>ie.</i> routage des cellules entre elles) avec route_design
11	Effectuer des check avec par exemple check_connectivity ou check_drc
12	Faire l'extraction de parasites avec extract_rc et l'enregistrer avec write_sdf
13	Générer des rapports avec par exemple time_design
14	Exporter le design final avec write_db

Table 5: Etapes clés réalisées lors de l'implémentation physique

Note: Encore une fois, les étapes restent inchangées d'un flow à l'autre mais pas les commandes pour les

réaliser! Entre, ces étapes, il est d'usage de générer des rapports et des saves de a database (qui permettent par la suite de reprendre à partir d'une save plutôt que depuis le début)

4.1. Entrées et sorties

Voici les entrées/sorties usuelle de cette phase de PnR :

Élément	Exemple	Rôle
RTL synthétisé	ripple_carry_4_netlist.v	Sortie de Genus : RTL mappé avec le pdk
Contraintes synthétisée	ripple_carry_4_sdc.sdc	Sortie de Genus : contraintes temporelle mappées
Variables du pdk	PNR_ASPECT_RATIO PNR_BOTTOM_LAYER	Spec. du pdk pour savoir quelle règles respecter
MMMC	constraints/mmmc.tcl	Pour les corner (voir plus loin)
Liberty	typical.lib	Infos sur les stdcell pour le MMMC
cds.lib	cds.lib	Entre autre pour les bases OA (voir plus loin)
[optionnel] - Checkpoints	database format LEF ou OA	Pour pouvoir repartir d'une étape
Fichiers SDF	ripple_carry_4_delay.sdf	Extraction de parasites poste route
Rapports	report_timing.rpt, report_area.rpt, report_qor.rpt	Preuves à examiner
design exporté	ripple_carry_4_OA	design routé sous un format LEF ou OA

Table 6: **entrées, sorties** de la minimales du placement routage

4.1.1. Les MMMC

TODO: MMMC explication a revoir/edulcorer

MMMC pour Multi Mode Multi Corner est l'analyse qui permet de savoir comment répond notre design avec des erreurs de fabrications, en température etc..

Avant de pouvoir envoyer un design en production, il faut d'abord s'assurer qu'il va fonctionner même si tel transistor est un peu plus lent, tel autre un peu plus rapide etc. C'est l'objectif du MMMC : faire des estimations de temps de passage dans les standard cell en fonction de chaque corner.

Les deux notions essentielles à comprendre à ce stade sont le setup et hold:

Setup Time (Temps d'Établissement)

- **Définition** : Temps minimum avant le front montant du clock pendant lequel la donnée doit être stable pour être correctement capturée par un registre.

- **Violation** : Si la donnée arrive trop tard ou pas assez en avance → Échec de capture (métastabilité).
- **Corner utilisé** : Worst-Case (WC) → Délais maximaux (pire cas pour le setup).

Hold Time (Temps de Maintien)

- **Définition** : Temps minimum après le front montant du clock pendant lequel la donnée doit rester stable pour éviter la métastabilité.
- **Violation** : Si la donnée change trop tôt → Échec de maintien (valeur instable).
- **Corner utilisé** : Best-Case (BC) → Délais minimaux (meilleur cas pour le hold).

4.1.2. Le format Open Access (OA) VS LEF

Par défaut, Innovus utilise un format LEF pour écrire et lire les données physiques (qu'on pourrait vulgariser par "vue layout"). C'est cette extension qui va décrire les tailles des cellules, ou sont les pistes métalliques, quelle largeur etc...

Pour faire l'implémentation physique on peut donc tout à fait donner à Innovus les vues .lef de la techno (qu'on peut retrouver dans le pdk). On entend ici un fichier qui va décrire le "layout" de chaque standard cell. Il va les comprendre et, à son tour, est capable de générer des .lef avec le floorplan complet, avec le design routé (ou son les cellules + comment elles sont reliées), avec le CTS (comment arrive le signal de clock ie. sorte de routage spécifique à la clock car c'est l'élément le plus sensible) etc.

Mais il y a un gros point noir à cette méthode : les fichiers .lef ne sont pas directement lisibles par **virtuoso** (qui permet notamment de faire le LVS et de mettre le guardring (cf. plus loin)). Il est donc recommandé de passer par le format **open access**. Ce format est lisible par Innovus (bien qu'il fasse une conversion en interne vers un format .lef, cette opération est transparente pour les utilisateur) **et** par Virtuoso.

Pour qu'Innovus puisse fonctionner **il a besoin d'un fichier cds.lib** dans le dossier depuis lequel il est exécuté. Dans ce cds.lib (pour cadence library) on retrouve tous les chemins vers les librairies utilisées par cadence (que ça soit les librairies du pdk ou bien les librairies des projets/design). Dans ce cds.lib est donc défini à un endroit le dossier des standard cell du pdk par exemple :

```
1  DEFINE ihp130_stdcell le/chemin/absolu/vers/le/pdk lib
```

Quand on va lire les librairies d'implémentation physique avec Innovus on va alors non plus lui donner des lef mais directement le nom *ihp130_stdcell* qu'il va pouvoir retrouver grâce au cds.lib (avec l'option -oa_ref_lib pour lui indiqué qu'on travaille avec des bases OA).

Par la suite, on va enregistrer la base open access, non plus en format .lef mais, en format oa avec une commande du type :

```
1  write_db -oa_lib_cell_view "$oaLibDir $oaLibName imported" tcl
```

Cette commande va créer dans le répertoire d'exécution un dossier \$oaLibDir dans lequel on va retrouver une cellule (un design) \$oaLibName contenant une vue "imported" (cette vue peut être n'importe quoi mais il y a des noms usuels comme floorplan, placed, routed, pré_CTS, post_CTS etc.)

Si d'aventure, on voudrait repartir d'une database OA précédemment sauvegardée il suffit de taper :

```
@innovus 1> read_db -oa_lib_cell_view "$oaLibDir $oaLibName placed"
read_db -oa_lib_cell_view "RIPPLE_CARRY_4_OA ripple_carry placed"
```

→ Commande bash

TODO: erreur lecture

Note : OA ou LEF

Il est important de noter ici qu'il faut faire un choix entre travailler en format LEF ou en format OA. On ne peut pas faire les deux en même temps.

4.2. Niveau 1 : script minimal

Dossier : flow/03_pnr/01_pnr_minimal/

Comme pour ma synthèse, l'appel direct à l'outil n'est pas possible car il y a trop d'étapes et de paramètres. Le niveau 1 donne un script minimal de placement routage dans lequel on laisse travailler Innovus **en automatique** sur une grande partie des tâches. De même que pour la synthèse, ici c'est suffisant car notre circuit est très simple mais dans le cas de plus gros circuits ça ne suffit pas → il faut donner davantage d'indications à Innovus (avec des commande du type **set_db** ou des paramètres dans les appels des fonctions)

Pour commencer on va se placer dans le dossier de travail. Depuis la racine :

```
cd flow/03_pnr/01_pnr_minimal/workdir
```

→ Commande bash

Pour lancer innovus avec le fichier :

```
# comme pour genus
innovus -stylus -files ../pnr_minimal.tcl -log logs/innovus
# Pour ouvrir la gui après coup
innovus -stylus &
```

→ Commande bash

Note importante : Fonctionnement de Innovus

Comme Genus, Innovus se lance grâce à la commande :

```
innovus -stylus
```

→ Commande bash

Si on spécifie un fichier d'exécution comme

```
innovus -stylus -file monfichier.tcl
```

→ Commande bash

alors innovus va s'ouvrir en mode stylus puis exécuter **ligne par ligne** le script. On pourrait tout aussi bien ouvrir Innovus avec la première commande et copier coller les commandes dans l'interpréteur Innovus ligne par ligne (ou bloc d'eligne par bloc de ligne) et ça en reviendrait au même.

Pour ouvrir la vue graphique d'Innovus, il faut d'abord avoir initialisé la database d'innovus (i.e. avoir atteint la ligne "init_design" ou être reparti d'une sauvegarde OA) puis on peut simplement faire :

```
@innovus 1> gui_show
```

→ Commande bash

```
@innovus 1> gui_fit
```

4.3. Notions importantes à connaître

4.3.1. Niveau de métaux utilisés

Souvent, les standard cell utilisent le premier niveau de métal pour le routage. Il est donc usuel d'indiquer à Innovus de commencer le routage avec le niveau de métal suivant. Il est par ailleurs courant de réserver le deuxième niveau de métal à l'arbre d'horloge. On va donc commencer le routage du reste au niveau 3.

4.3.2. skew

Le **skew** est le retard d'arrivée du front montant de l'horloge du à la propagation. On pourrait penser de prim abord que ce retard est indésirable mais il est parfois très pratique (et même induit exprès par innovus)! Par exemple, sur la **Figure 13** on peut voir que les data vont de gauche à droite. Si la clk arrivait également de gauche à droite à ce moment là, la valeur pourrait passer de la bascule 1 à 2 mais serait bloquée ensuite car le front d'horloge serait passé avant que le temps de setup soit respecté.

TODO: revoir ce truc

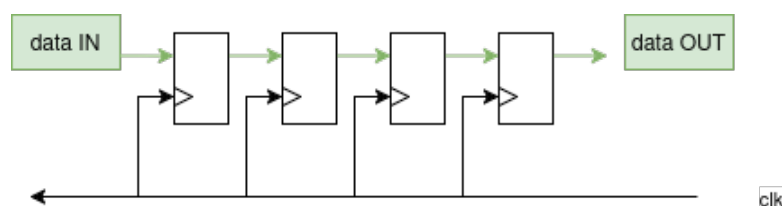


Figure 13: Exemple typique de l'utilité du skew

4.4. Ouverture du design dans Virtuoso

Une fois qu'on a correctement exporté le design dans une base Open Access, on peut l'ouvrir pour la visualiser dans virtuoso.

Pour se faire, il faut démarrer virtuoso dans le répertoire habituel et bien penser à inclure dans le cds.lib de ce répertoire le lien vers le cds.lib dans lequel Innovus a mis le chemin vers sa base OA en rajoutant la ligne

```
1 INCLUDE /chemin/vers/mon/cds.lib
```

lib

De cette manière, virtuoso va immédiatement trouver la base OA dans son library manager comme on peut le voir **Figure 14**.

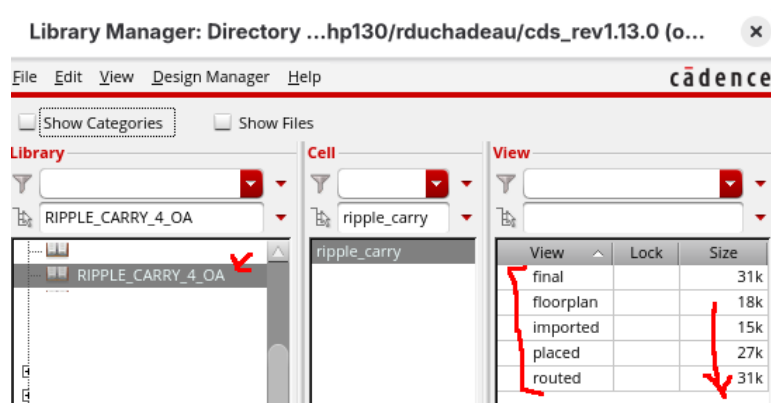


Figure 14: Virtuoso : ouverture base OA

On remarque alors que toutes les vues qu'on a enregistré y figurent et que leurs taille augmente petit à petit (normal car au fur et à mesure des étapes on rajoute des éléments). On peut par exemple ouvrir la vue "routed" et on voit alors :

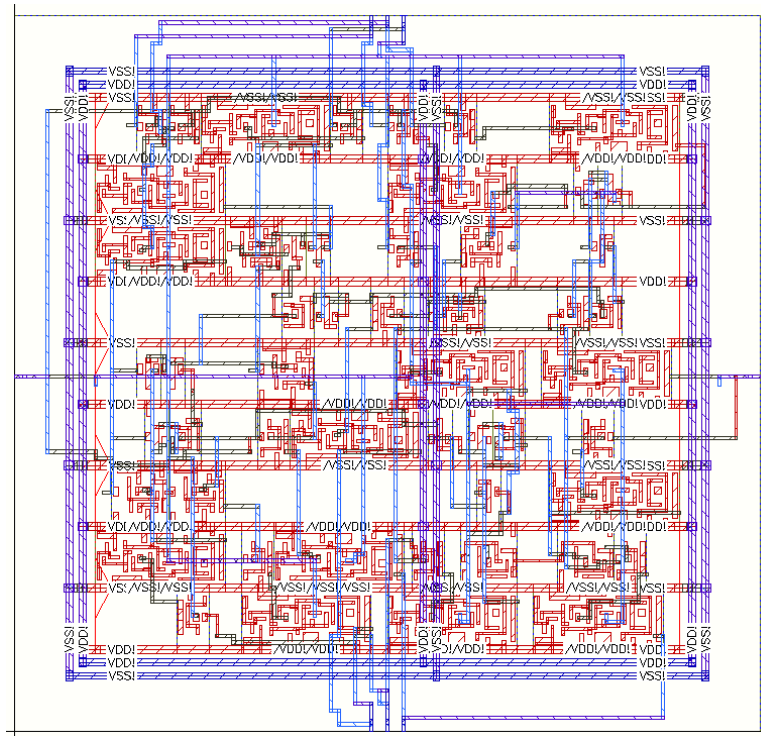


Figure 15: Vue complete de la puce ripple_carry_4

Super! On a une puce “complete”. En réalité, il y aurait plusieurs choses à rajouter ou modifier comme le filling (rajout de dummy cells entre les cellules “fonctionnelles” qu’on voit sur la **Figure 15**) ou bien les rails d’alim qui, ici, n’ont pas en top métal car le circuit aurait été trop petit en comparaison et qui est trop proche de la puce.

On remarque par ailleurs que les cellules apparaissent relativement vides par rapport à ce à quoi on pourrait s’attendre. C’est normal, en réalité, les cellules standard sont exportées en format “**abstract**” plus léger et pratique pour travailler en numérique. Si la vue layout est disponible dans la techno cible on peut alors changer les cellules abstract en layout en faisant : **tool** → **find/replace** puis :

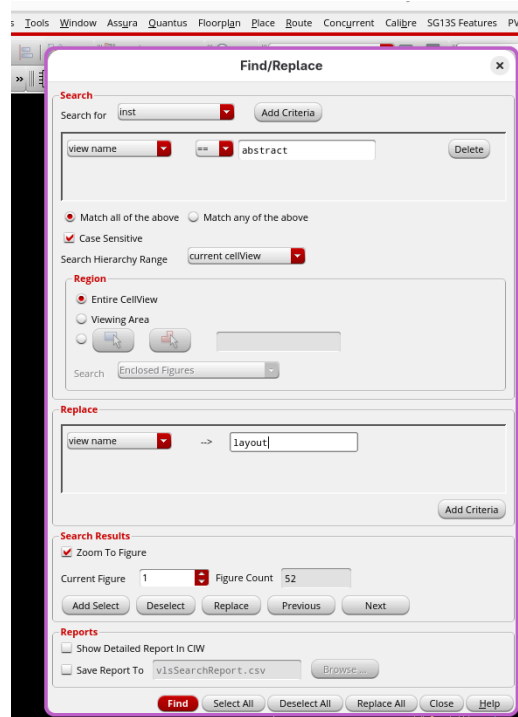


Figure 16: Comment remplacer la vue abstract par layout

On peut observer que les cellules se sont “remplies” et que les différentes couches de métaux qui les composent apparaissent (cf. Figure 17).

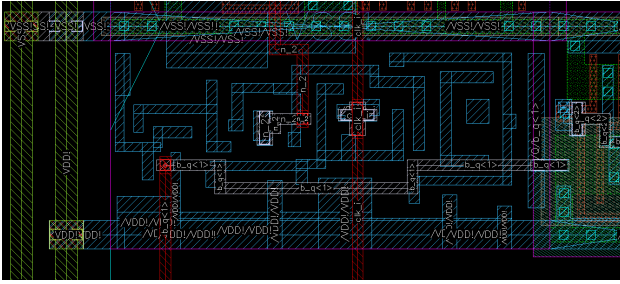
Récap : Vue Layout vs vue abstract

Quand on ouvre le design dans virtuoso, on remarque que les standard cell sont en vue abstract et non en vue layout. La vue abstract est en fait une **vue layout simplifiée** ou apparaissent:

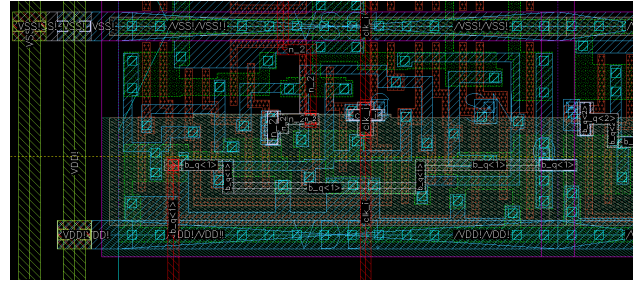
1. Les connections
2. Le niveau de métal 1
3. Une zone ou il est interdit de passer au dessus.

Cette vue permet d'effectuer une routage fonctionnel tout en restant plus légère à lire et en preservant la propriété intellectuelle. C'est d'ailleurs pour ce dernier point que certain pdk ne contiennent pas de vue layout sur leurs standard cell.

Pour envoyer un circuit en fonderie il est **impératif** de fournir la vue layout si elle est disponible. Si elle ne l'est pas, il faut bien stipuler au fondeur qu'il va devoir faire le changement lui même.



(a) Vue abstract



(b) Vue layout

Figure 17: Différence entre la vue abstract et la vue layout

Ansi, notre puce “finale” ressemble à ceci :

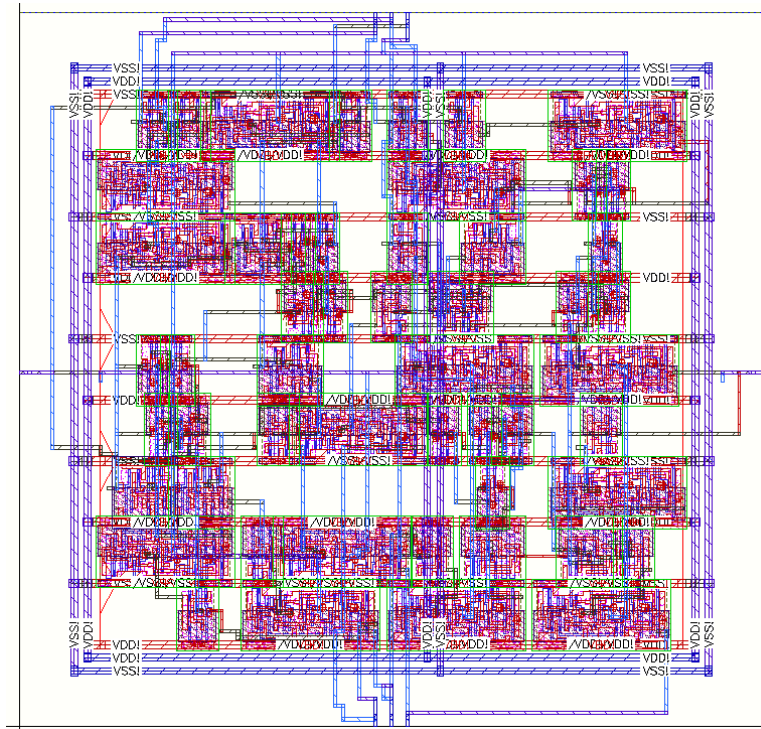


Figure 18: Vue complete de la puce ripple_carry_4 après changement vers vue layout

5. Etape 4 : Simulation gate-level

Dossier : flow/01_simulation/02_simulation_gate_level

Une fois que le placement routage est terminé, il nous reste à vérifier que le design “final” est toujours fonctionnel. Pour ce faire on va refaire un étape de simulation avec Xcellium mais avec deux changements majeurs :

1. On utilise cette fois le fichier RTL donné par Innovus avec notre module contenant les cellules standard. Pour que xrun puisse comprendre à quoi font référence les modules de cellules standard que Innovus a mis, il faut également lui indiquer les modèles comportementaux de ces cellules (il faut par exemple que quelque part xrun comprenne que le module OR3JLTX1 effectue une fonction “OU”). Pour ce faire, on va également rentrer en option les fichiers du PDK ou le comportement des cellules standard est décrit (facile à trouver c’est un fichier du type stdcell.v).
2. On donne également à xrun le fichier .sdf généré par Innovus. Ce fichier spécifie point par point les retards dans les signaux à cause des ralentissements causés par le chemin de la clock (clock tree) ou bien par les parasites.

Dans notre cas, la commande complète ressemble à :

```
xrun -64bit \  
-sv \  
-access +rwc \  
-gui \  
-timescale 1ns/1ps \  
/share/Pdk/IHP/SG13S/ixc013_stdcell/verilog/ixc013_stdcell.v \  
/share/Pdk/IHP/SG13S/ixc013_stdcell/verilog/ixc013_primitives.v \  
flow/03_pnr/01_pnr_minimal/workdir/outputs/ripple_carry_4.routed.v \  
dut/rtl/tb_ripple_carry_4.sv \  
-sdf_cmd_file flow/01_simulation/02_simulation_gate_level/sdf_cmd_file -mess
```

Note : nomenclature

On appelle cette étape la simulation gate level et non “extract” comme on pourrait le faire en analogique car ce terme est réservé à virtuoso.

5.1. Attention au testbench

On a vu que la simulation gate level utilisait le même testbench que pour la simulation RTL. Mais attention, quelque chose a changé → on a rajouté des délais et, entre autres, des **imprécisions sur l’arrivée de**

l'horloge. Ainsi, si Pour vérifier que notre design fonctionne on lit la valeur de sortie exactement au front d'horloge, on a de grandes chances qu'elle ne soit pas encore arrivée!!!

C'est pourquoi il est **primordial** de prendre ce délais en compte en introduisant par exemple un temps mort avant la lecture comme on peut le voir dans le testbench :

```

...
54     task automatic check_result(
55         input integer test_id,
56         input logic [4:0] expected,
57         input string detail
58     );
59     #1;
60
61     if ({cout_o, sum_o} !== expected) begin
62         errors = errors + 1;
63
64         $display(
65             "[TEST_FAILED] TEST %0d : %s | resultat=%0d attendu=%0d",
66             test_id,
67             detail,
68             {cout_o, sum_o},
69             expected
70         );
71     end
72     else begin
73         $display(
74             "[TEST_PASS] TEST %0d : %s | resultat=%0d",
75             test_id,
76             detail,
77             {cout_o, sum_o}
78         );
79     end
80
81     endtask

```

Extrait de tb_ripple_carry_4.sv

5.2. Comment savoir si le fichier sdf est bien pris en compte dans la simulation ?

Pour savoir si on simule bien le rtl placé routé c'est simple : on ne pointe que vers ce fichier et non plus vers les .sv d'avant. Pour savoir si xrun prend bien en compte le fichier sdf pour les parasites c'est moins évident. Quand ouvre la waveform, il suffit d'observer un petit décalage entre la clock et un signal synchrone. Si les signaux sont légèrement désynchronisés c'est gagné! Par exemple, on peut voir sur la **Figure 19** que le SDF est correctement pris en compte car cout_o est sensé monter en même temps que la clock.

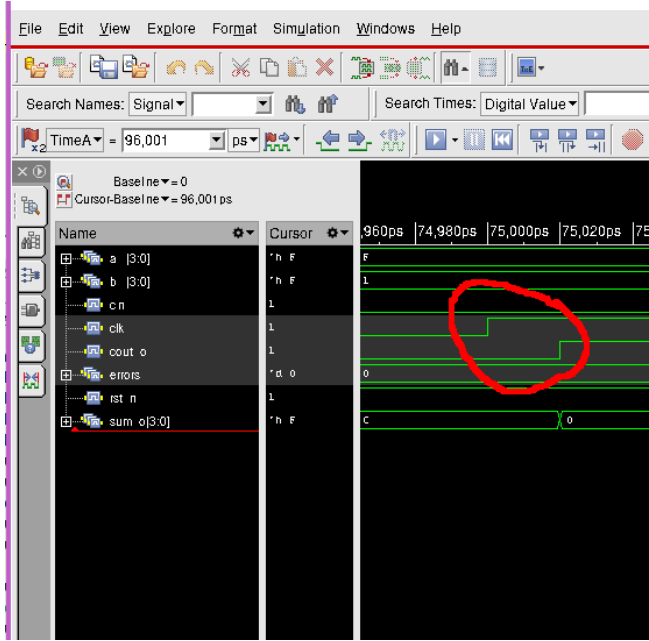


Figure 19: Simulation gate level avec SDF pris en compte